

✓ *Import Data*

```
import pandas as pd
"""
from google.colab import drive
drive.mount("/content/drive",force_remount=True)
path="/content/drive/My Drive/data/digit.csv"
df =pd.read_csv(path)
"""
#df = pd.read_csv("digit.csv")
data=pd.read_csv("digit.csv",header=None)
```

✓ Try to predict using simple MLP

```
X = data.values

from sklearn.model_selection import train_test_split
X_train , X_test, Y_train, Y_test=train_test_split(X[:, :784],X[:, 784],

# ignore the last column since it is label
```

```
data.shape
```

```
(500, 785)
```

```
from sklearn.neural_network import MLPClassifier
mlp = MLPClassifier(random_state=1,verbose=1)
mlp.fit(X_train, Y_train)
```

```
from sklearn.linear_model import LinearRegression
```

Show score of the prediction

```
print('The score for training data is', mlp.score(X_train,Y_train))  
print('The score for test data is',mlp.score(X_test,Y_test))
```

```
The score for training data is 1.0  
The score for test data is 0.74
```

```
print('Data Size', X.shape)  
print(X)
```

```
Data Size (500, 785)  
[[0 0 0 ... 0 0 0]  
 [0 0 0 ... 0 0 0]  
 [0 0 0 ... 0 0 0]  
 ...  
 [0 0 0 ... 0 0 9]  
 [0 0 0 ... 0 0 9]  
 [0 0 0 ... 0 0 9]]
```

✓ PCA

```
import numpy as np

# we try to find the Eigen vector and Eigen value
X_std=X[:, :-1]
mean_vec = np.mean(X_std, axis=0) # we try to find the mean of each di
```

```
print(X_std.shape)
print(mean_vec.shape)
```

```
(500, 784)
(784,)
```

```
cov_mat = (X_std - mean_vec).T.dot((X_std - mean_vec)) / (X_std.shape[0]-1)
print('Covariance matrix \n%s' %cov_mat)
print('NumPy covariance matrix: \n%s' %np.cov(X_std.T))
cov_mat = np.cov(X_std.T)
from numpy import linalg as LA
values, vectors = LA.eig(cov_mat)

import matplotlib.pyplot as plt
plt.plot(np.sort(values)[::-1],'.')
plt.tick_params(axis='y',length=6, width=2)

# Y axis = Eigen value, the higher = more meaningful
```

```
print(values.shape)
print(vectors.shape)
```

```
(784,)
(784, 784)
```

You can find Covariance matrix manually or use Library

```
from sklearn.decomposition import PCA
pca = PCA(.95)
# We calculate the sum of all the eigen value
# .95 is the percent we want of the total sum
# from highest vector only include until reach the percentage in this
X_new=pca.fit_transform(X_std)
eig_vals=pca.explained_variance_

print('Eigenvectors \n%s' %pca)
print('\nEigenvalues \n%s' %eig_vals)
```

```
Eigenvectors
PCA(n_components=0.95)
```

```
Eigenvalues
[345461.47812618 250042.1076638 222144.04019841 198563.31989173
 172166.12259708 159213.2261283 111376.78115598 99451.49059151
 92369.38200562 79878.70193572 77003.27055431 62799.79128478
 61098.14234933 57633.83112974 53608.24933423 52586.98577648
 47317.9495977 46061.2386965 38506.37088837 36396.34648642
 34082.67975417 34730.94100479 34083.74576035 31469.09314083]
```

29815.32361248	28191.17179626	27130.52366731	26721.66358086
24383.62134238	23889.75530367	22362.53656761	21442.35883883
19975.33515133	18976.7397448	18785.34588143	17677.40496539
17203.95784605	16661.71538269	16349.86501195	15016.11022748
14215.6209982	14065.90284823	13976.49130551	13409.05042521
12749.90260065	12363.01313157	12098.36077756	11873.81919225
11638.24003423	10824.04498684	10428.15086058	9920.32856987
9691.88373752	9387.66095846	9125.75567998	8982.28587021
8652.07782825	8382.1849651	7883.54970186	7798.71757933
7369.12460534	7349.32020097	7131.94783583	7018.03299712
6809.19588722	6609.23899937	6396.05263566	6374.27135366
6091.76266957	6013.1604701	5937.23466164	5700.53110204
5652.2009494	5513.08455857	5307.2065358	5084.92587974
5074.31227038	4988.25301827	4692.99014795	4638.19435954
4585.10570642	4360.052036	4300.60750718	4272.38028584
4180.64406225	4064.51736737	3929.14997514	3877.59002446
3798.05145555	3708.87600293	3678.81321363	3663.30342427
3605.38128277	3553.93256143	3412.95212396	3330.33945283
3243.08285807	3229.44602259	3158.44057754	3113.51622679
3090.60445622	2972.6175148	2912.32514107	2822.54781953
2808.10175722	2767.13513815	2681.44856803	2628.27396935
2592.06580324	2532.4078472	2486.10791186	2450.20715703]

```
import matplotlib.pyplot as plt
plt.plot(eig_vals, '.')
plt.tick_params(axis='y', length=6, width=2)
```

```
plt.show()
```

```
eig_vecs=pca.components_  
Projdata=np.dot((X_std-mean_vec),eig_vecs.T)  
plt.clf()  
  
digit=np.arange(50)  
  
plt.plot(Projdata[digit,1],Projdata[digit,2], 'x', color='g')  
digit=digit+200  
plt.plot(Projdata[digit,1],Projdata[digit,2], 'x', color='r')  
  
plt.show()
```

```
eig_vecs.shape
```

```
(112, 784)
```

```
eig_vecs[0]
```

```
array([-0.00000000e+00, -0.00000000e+00, -0.00000000e+00,  
       -1.38777878e-17,  
         3.46944695e-18,  1.73472348e-18, -0.00000000e+00,  
       -0.00000000e+00,  
        -0.00000000e+00, -1.35525272e-20, -0.00000000e+00,  
       -0.00000000e+00,  
        -0.00000000e+00, -0.00000000e+00, -0.00000000e+00,  
       -0.00000000e+00])
```

```

-0.00000000e+00, -0.00000000e+00, -0.00000000e+00,
-0.00000000e+00,
-0.00000000e+00, -0.00000000e+00, -0.00000000e+00,
-0.00000000e+00,
-0.00000000e+00, -0.00000000e+00, -0.00000000e+00,
-0.00000000e+00,
5.73971851e-42, -1.43492963e-42, 4.03573958e-42,
-0.00000000e+00,
-0.00000000e+00, -0.00000000e+00, -0.00000000e+00,
-0.00000000e+00,
-0.00000000e+00, -0.00000000e+00, -0.00000000e+00,
-0.00000000e+00,
-0.00000000e+00, 3.24528725e-04, 6.27617873e-04,
3.90403353e-04,
7.43189991e-04, 1.86526116e-04, -0.00000000e+00,
-0.00000000e+00,
-0.00000000e+00, -0.00000000e+00, -0.00000000e+00,
-0.00000000e+00,
-0.00000000e+00, -0.00000000e+00, -0.00000000e+00,
-0.00000000e+00,
-0.00000000e+00, -0.00000000e+00, -0.00000000e+00,
-0.00000000e+00,
-0.00000000e+00, -0.00000000e+00, -0.00000000e+00,
-0.00000000e+00,
-0.00000000e+00, -0.00000000e+00, 5.80276996e-06,
3.21433224e-05,
-1.59739971e-04, 1.69210979e-04, 9.42535232e-04,
3.18029827e-03,
4.16176439e-03, 1.72934618e-03, 1.48493262e-03,
1.29094599e-03,
3.69794253e-04, -0.00000000e+00, -0.00000000e+00,
-0.00000000e+00,
-0.00000000e+00, -0.00000000e+00, -0.00000000e+00,
-0.00000000e+00,
-0.00000000e+00, -0.00000000e+00, -0.00000000e+00,
-0.00000000e+00,
-0.00000000e+00, -0.00000000e+00, -2.06424981e-05,
-7.24527404e-05,
-2.08051265e-04, -2.48142290e-04, -2.01240672e-04,
1.30523617e-03,
4.02676983e-03, 3.96905362e-03, 6.15205729e-03,
9.92527455e-03,
1.17838434e-02, 1.10340316e-02, 7.97749277e-03,
4.67203659e-03,
1.98297998e-03, 6.40185988e-04, -3.71777926e-04,
-0.00000000e+00,
-0.00000000e+00, -0.00000000e+00, -0.00000000e+00,
-0.00000000e+00,
-0.00000000e+00, -0.00000000e+00, -0.00000000e+00,
-0.00000000e+00,

```

```
import matplotlib.pyplot as plt
```

```
Projdata50=np.dot((X_std-).T,eig_vecs[:,1:51])
```

size(Projdata50)

```
import matplotlib.pyplot as plt
```

```
Projdata50=np.dot((X_std-mean_vec),eig_vecs[:400,:].T)
```

```
recon50=np.dot(Projdata50, eig_vecs[:400,:]) + mean_vec  
recon50.shape
```

```
(500, 784)
```

```
plt.imshow(np.reshape(recon50[1],[28,28]))  
plt.show()
```

```
# Projdata100=np.dot((X_std-mean_vec).T,eig_vecs[:,1:101])  
# recon100=np.dot(eig_vecs[:,1:101], Projdata100.T)+mean_vec  
# plt.imshow(np.reshape(recon100[266,:],[28,28]))  
# plt.show()
```

```
eig_vecs.sort
```

```
<function ndarray.sort>
```

```
nprint(eig_vecs)
```



```
pca.transform(images)
```

```
[[ -0.00000000e+00 -0.00000000e+00 -0.00000000e+00 ... -0.00000000e+00
  -0.00000000e+00 -0.00000000e+00]
 [ -1.24900090e-16  9.02056208e-17  1.17961196e-16 ... -0.00000000e+00
  -0.00000000e+00 -0.00000000e+00]
 [  2.49800181e-16 -1.38777878e-17 -2.77555756e-17 ...  0.00000000e+00
   0.00000000e+00  0.00000000e+00]
 ...
 [ -1.26634814e-16 -1.50920942e-16 -2.60208521e-17 ...  0.00000000e+00
   0.00000000e+00  0.00000000e+00]
 [ -3.98986399e-17 -3.20923843e-16 -3.71230824e-16 ...  0.00000000e+00
   0.00000000e+00  0.00000000e+00]
 [  3.03576608e-18  4.02455846e-16  6.07153217e-17 ... -0.00000000e+00
  -0.00000000e+00 -0.00000000e+00]]
```

```
# This is an extremely inefficient function.
def pca_com(com, images):
    # percentage should be a decimal from 0 to 1
    pca = PCA(n_components=com)
    pca.fit(images)
    components = pca.transform(images)
    approxOriginal = pca.inverse_transform(components)
    return approxOriginal
```

```
pca = PCA(n_components=80)
pca.fit(X_std)
plt.plot(np.cumsum(pca.explained_variance_ratio_),'.')
plt.show()
```

```
plt.clf()
plt.figure(figsize=(20,4));

# Original Image (784 components)
plt.subplot(1, 6, 1);
plt.imshow(X_std[260,:].reshape(28,28),
           cmap = plt.cm.gray, interpolation='nearest',
           clim=(0, 255));
plt.xlabel('784 Components', fontsize = 12)
plt.title('Original Image', fontsize = 14);

# 10 principal components
plt.subplot(1, 6, 2);
plt.imshow(pca_com(10, X_std)[260].reshape(28, 28),
           cmap = plt.cm.gray, interpolation='nearest',
           clim=(0, 255));
plt.title('10 components', fontsize = 12);

# 20 principal components
plt.subplot(1, 6, 3);
plt.imshow(pca_com(20, X_std)[260].reshape(28, 28),
           cmap = plt.cm.gray, interpolation='nearest',
           clim=(0, 255));
plt.title('20 Components', fontsize = 12)

# 30 principal components
plt.subplot(1, 6, 4);
plt.imshow(pca_com(30, X_std)[260].reshape(28, 28),
           cmap = plt.cm.gray, interpolation='nearest',
           clim=(0, 255));
plt.title('30 Components', fontsize = 12)

# 40 principal components
plt.subplot(1, 6, 5);
plt.imshow(pca_com(40, X_std)[260].reshape(28, 28),
           cmap = plt.cm.gray, interpolation='nearest',
           clim=(0, 255));
plt.title('40 Components', fontsize = 12)

# 50 principal components
plt.subplot(1, 6, 6);
```

```
plt.imshow(pca_com(50, X_std)[260].reshape(28, 28),
            cmap = plt.cm.gray, interpolation='nearest',
            clim=(0, 255));
plt.title('50 components', fontsize = 12);
plt.show()
```

```
pca = PCA(.99)
pca.fit(X_std)
```

```
pca.n_components_
```

```
np.int64(227)
```

```
pca_com
```

```
plt.clf()
plt.figure(figsize=(20,4));

# Original Image (784 components)
plt.subplot(1, 6, 1);
plt.imshow(pca_com(40, X_std)[260].reshape(28, 28))
```

```
plt.imshow(pca_com(60, X_std)[260].reshape(28, 28),
            cmap = plt.cm.gray, interpolation='nearest',
            clim=(0, 255));

plt.title('60 components', fontsize = 14);

# 10 principal components
plt.subplot(1, 6, 2);
plt.imshow(pca_com(70, X_std)[260].reshape(28, 28),
            cmap = plt.cm.gray, interpolation='nearest',
            clim=(0, 255));
plt.title('70 components', fontsize = 12);

# 20 principal components
plt.subplot(1, 6, 3);
plt.imshow(pca_com(80, X_std)[260].reshape(28, 28),
            cmap = plt.cm.gray, interpolation='nearest',
            clim=(0, 255));
plt.title('80 Components', fontsize = 12)

# 30 principal components
plt.subplot(1, 6, 4);
plt.imshow(pca_com(90, X_std)[260].reshape(28, 28),
            cmap = plt.cm.gray, interpolation='nearest',
            clim=(0, 255));
plt.title('90 Components', fontsize = 12)

# 40 principal components
plt.subplot(1, 6, 5);
plt.imshow(pca_com(100, X_std)[260].reshape(28, 28),
            cmap = plt.cm.gray, interpolation='nearest',
            clim=(0, 255));
plt.title('100 Components', fontsize = 12)

# 50 principal components
plt.subplot(1, 6, 6);
plt.imshow(pca_com(110, X_std)[260].reshape(28, 28),
            cmap = plt.cm.gray, interpolation='nearest',
            clim=(0, 255));
plt.title('110 components', fontsize = 12);
plt.show()
```

```
pca = PCA(200)
X_train_new=pca.fit_transform(X_train)
X_test_new=pca.transform(X_test)
```

```
mlp2 = MLPClassifier(random_state=1,verbose=1)
mlp2.fit(X_train_new, Y_train)
```

```
print('The score for training data is', mlp2.score(X_train_new,Y_train))
print('The score for test data is',mlp2.score(X_test_new,Y_test))
```

```
The score for training data is 1.0
The score for test data is 0.65
```

```
X_train_new.shape
```

```
(400, 200)
```

```
from sklearn.preprocessing import StandardScaler
scaler2 = StandardScaler()
X_train_new_std= scaler2.fit_transform(X_train_new)
X_test_new_std= scaler2.transform(X_test_new)

mlp4 = MLPClassifier(random_state=1,verbose=0)
mlp4.fit(X_train_new_std, Y_train)
print('The score for training data is', mlp4.score(X_train_new_std,Y_train))
print('The score for test data is',mlp4.score(X_test_new_std,Y_test))
```

```
The score for training data is 1.0
The score for test data is 0.66
```

```
scaler = StandardScaler()
X_train_std= scaler.fit_transform(X_train)
X_test_std= scaler.transform(X_test)

mlp3 = MLPClassifier(random_state=1,verbose=0)
mlp3.fit(X_train_std, Y_train)
print('The score for training data is', mlp3.score(X_train_std,Y_train))
print('The score for test data is',mlp3.score(X_test_std,Y_test))
```

```
The score for training data is 1.0
The score for test data is 0.82
```

Start coding or [generate](#) with AI.